

Foundations of Modern Web Architecture

From HTTP to FastAPI

Understanding Request-Response Lifecycle, Synchronous Processing, and Framework Internals

The Core of the Web: HTTP Protocol

- What is HTTP?
 - **H**ypertext **T**ransfer **P**rotocol An application-layer protocol designed for transmitting hypermedia documents.
 - **Stateless Protocol** Every request is executed independently. The server retains no memory of past requests without external session management (e.g., Cookies, JWT tokens).

The Core of the Web: HTTP Protocol

- The Request-Response Model
 - **Client (User-Agent)** Initiates a connection, constructs a structured text message (**Request**), and sends it over TCP/IP.
 - **Server** Listens on a designated port (e.g., 80, 8080), processes the incoming data, and returns a structured text message (**Response**).

Anatomy of an HTTP Request & Response

- **HTTP Request Structure**

- **Request Line** Method (**GET**, **POST**, **PUT**, **DELETE**), URI (Path), and Protocol Version (e.g., HTTP/1.1).
- **Headers** Metadata containing Client capabilities (User-Agent), Content-Type (application/json), and authentication keys.
- **Body** (Payload) Raw data sent to the server (primarily used in POST and PUT operations).

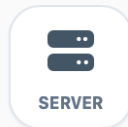
Anatomy of an HTTP Request & Response

- **HTTP Response Structure**

- **Status Line** Protocol Version, Status Code (e.g., **200 OK**, **404** Not Found, **500** Internal Server Error).
- **Headers** Server metadata (Server: uvicorn, Content-Length, Set-Cookie).
- **Body** The actual content returned to the client (HTML, JSON, Images).

LIVE SEQUENCE CORE

CONNECTING (TCP SYN)



1. HANDSHAKE

2. REQUEST

3. PROCESS

4. RESPONSE

5. TEARDOWN

TCP CONNECTION HANDSHAKE

Before HTTP can talk, TCP must handshake. A virtual socket pathway is built using synchronization (SYN) and acknowledgement (ACK) packets. No real data is transmitted yet.

OUTGOING NETWORK PACKET

Client Envelope

```
POST /student/6718381801/robot/robot_1/grab HTTP/1.1
Host: 127.0.0.1:8080
Content-Type: application/json
Content-Length: 18

{ "part": "A" }
```

Pending Request Send...

Contains the standard Request Line, static target headers, and a validated JSON entity.

INCOMING NETWORK PACKET

Server Envelope

```
HTTP/1.1 200 OK
Date: Sun, 19 Jul 2025 06:54:00 GMT
Server: uvicorn
Content-Type: application/json

{ "status": "success" }
```

Awaiting Response Pipeline...

Contains the status verification envelope (200 OK), server version identification, and output results.

LIVE SEQUENCE CORE

CLIENT SENDING ENVELOPE



1. HANDSHAKE 2. REQUEST 3. PROCESS 4. RESPONSE 5. TEARDOWN

→ CLIENT DISPATCHES HTTP REQUEST

The socket is open! The Client immediately serializes the HTTP metadata, headers, and payload into a single raw text packet, pushing it along the TCP pipeline.

OUTGOING NETWORK PACKET

Client Envelope

```
POST /student/6710301001/robot/robot_1/grab HTTP/1.1
Host: 127.0.0.1:8088
Content-Type: application/json
Content-Length: 16

{ "part": "A" }
```

i Contains the standard Request Line, static target headers, and a validated JSON entity.

INCOMING NETWORK PACKET

Server Envelope

```
HTTP/1.1 200 OK
Date: Sun, 19 Jul 2026 06:54:00 GMT
Server: uvicorn
Content-Type: application/json

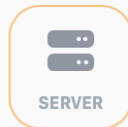
{ "status": "success" }
```

Awaiting Response Pipeline...

i Contains the status verification envelope (200 OK), server version identification, and output results.

LIVE SEQUENCE CORE

SERVER PROCESSING (WAITING)



1. HANDSHAKE

2. REQUEST

3. PROCESS

4. RESPONSE

5. TEARDOWN

⚙️ SERVER PROCESSING LOGIC

The HTTP packet is received by the server (FastAPI). It validates schema parsing (Pydantic), executes internal routing logic, and accesses backends. Meanwhile, the client sits completely idle, blocking other sync requests.

OUTGOING NETWORK PACKET

Client Envelope

```
POST /student/6710301001/robot/robot_1/grab HTTP/1.1
Host: 127.0.0.1:8088
Content-Type: application/json
Content-Length: 16

{ "part": "A" }
```

ⓘ Contains the standard Request Line, static target headers, and a validated JSON entity.

INCOMING NETWORK PACKET

Server Envelope

```
HTTP/1.1 200 OK
Date: Sun, 19 Jul 2025 06:54:00 GMT
Server: uvicorn
Content-Type: application/json

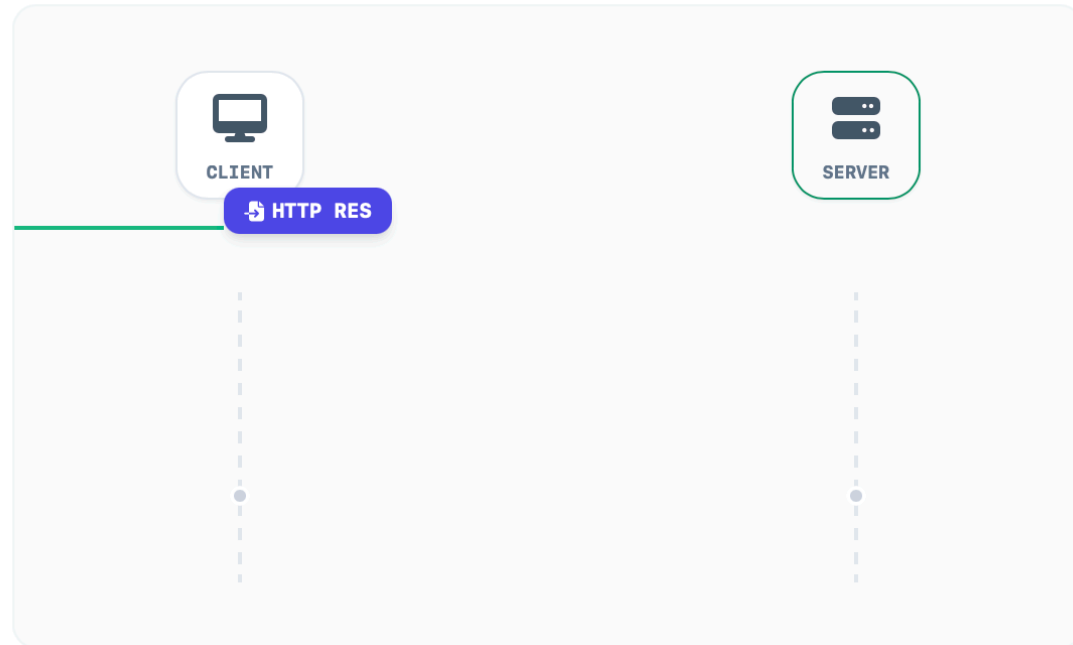
Processing Transaction...

{ "status": "success" }
```

ⓘ Contains the status verification envelope (200 OK), server version identification, and output results.

LIVE SEQUENCE CORE

SERVER TRANSMITTING DATA



1. HANDSHAKE 2. REQUEST 3. PROCESS 4. RESPONSE 5. TEARDOWN

← SERVER RETURNS HTTP RESPONSE

Server finishes calculations and packages output into an HTTP Response. It sends this envelope back along the same TCP channel.

OUTGOING NETWORK PACKET

Client Envelope

```
POST /student/6710301001/robot/robot_1/grab HTTP/1.1
Host: 127.0.0.1:8088
Content-Type: application/json
Content-Length: 16

{ "part": "A" }
```

i Contains the standard Request Line, static target headers, and a validated JSON entity.

INCOMING NETWORK PACKET

Server Envelope

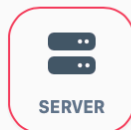
```
HTTP/1.1 200 OK
Date: Sun, 19 Jul 2026 06:54:00 GMT
Server: uvicorn
Content-Type: application/json

{ "status": "success" }
```

i Contains the status verification envelope (200 OK), server version identification, and output results.

LIVE SEQUENCE CORE

SOCKET TEARDOWN (CLOSED)



1. HANDSHAKE

2. REQUEST

3. PROCESS

4. RESPONSE

5. TEARDOWN

✂ TCP SOCKET IMMEDIATELY TORN DOWN

CRITICAL LESSON: Once the client receives the payload, the transaction is complete. The TCP socket is immediately disconnected and destroyed (or recycled to a pool). No standing state connection is preserved. STATELESSNESS ACCOMPLISHED.

OUTGOING NETWORK PACKET

Client Envelope

```
POST /student/6718381881/robot/robot_1/grab HTTP/1.1
Host: 127.0.0.1:8080
Content-Type: application/json
Content-Length: 16

{ "part": "A" }
```

Socket Terminated

Contains the standard Request Line, static target headers, and a validated JSON entity.

INCOMING NETWORK PACKET

Server Envelope

```
HTTP/1.1 200 OK
Date: Sun, 19 Jul 2026 06:54:00 GMT
Server: uvicorn
Content-Type: application/json

{ "status": "success" }
```

Socket Terminated

Contains the status verification envelope (200 OK), server version identification, and output results.

Traditional Web Servers vs. Threading Limits

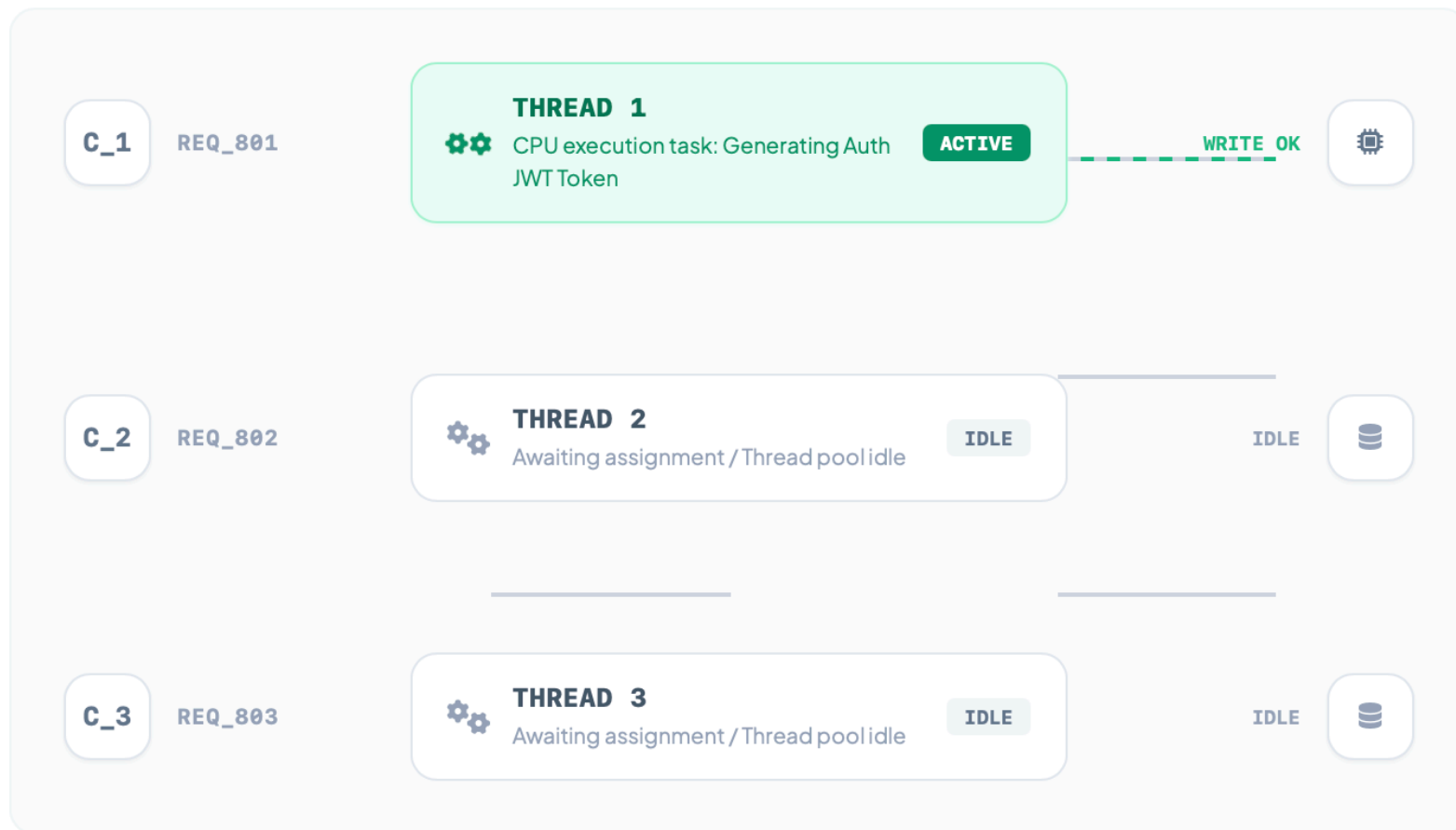
- **How Synchronous Servers Work (Traditional I/O)**

- **One Thread Per Connection** When an HTTP request arrives, the OS assigns a specific thread to handle it.
- **The Blocking Bottleneck** If the thread performs a heavy database query or waits for disk I/O, the entire thread stalls. It enters a "Blocked" state, wasting CPU cycles while idling.

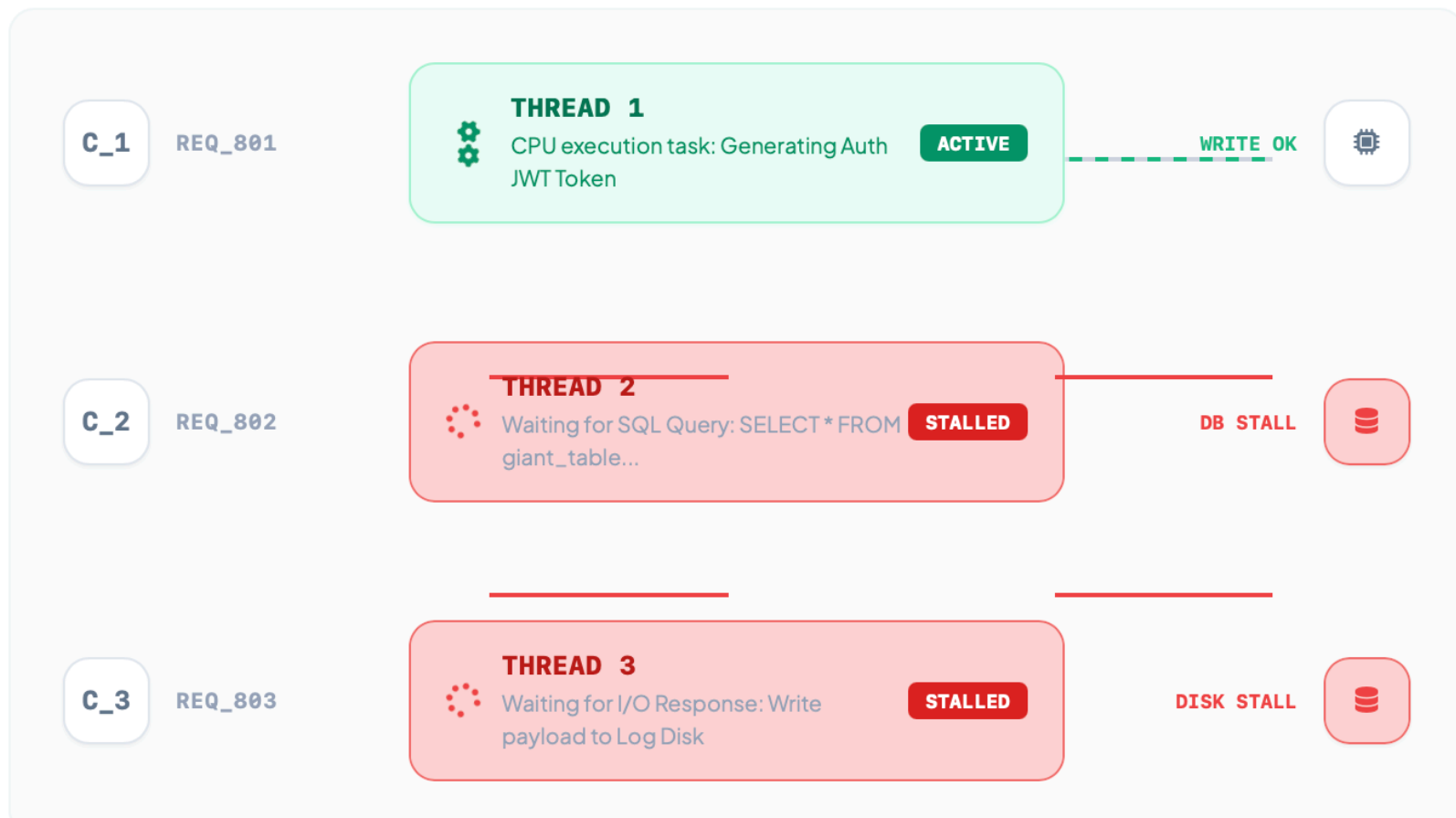
Traditional Web Servers vs. Threading Limits

- **The Scalability Wall**

- Operating system threads are expensive (consuming roughly **1-8 MB of RAM per thread**).
- Handling 1,000 concurrent requests synchronously requires 1,000 threads, quickly exhausting server memory and causing heavy context switching overhead.



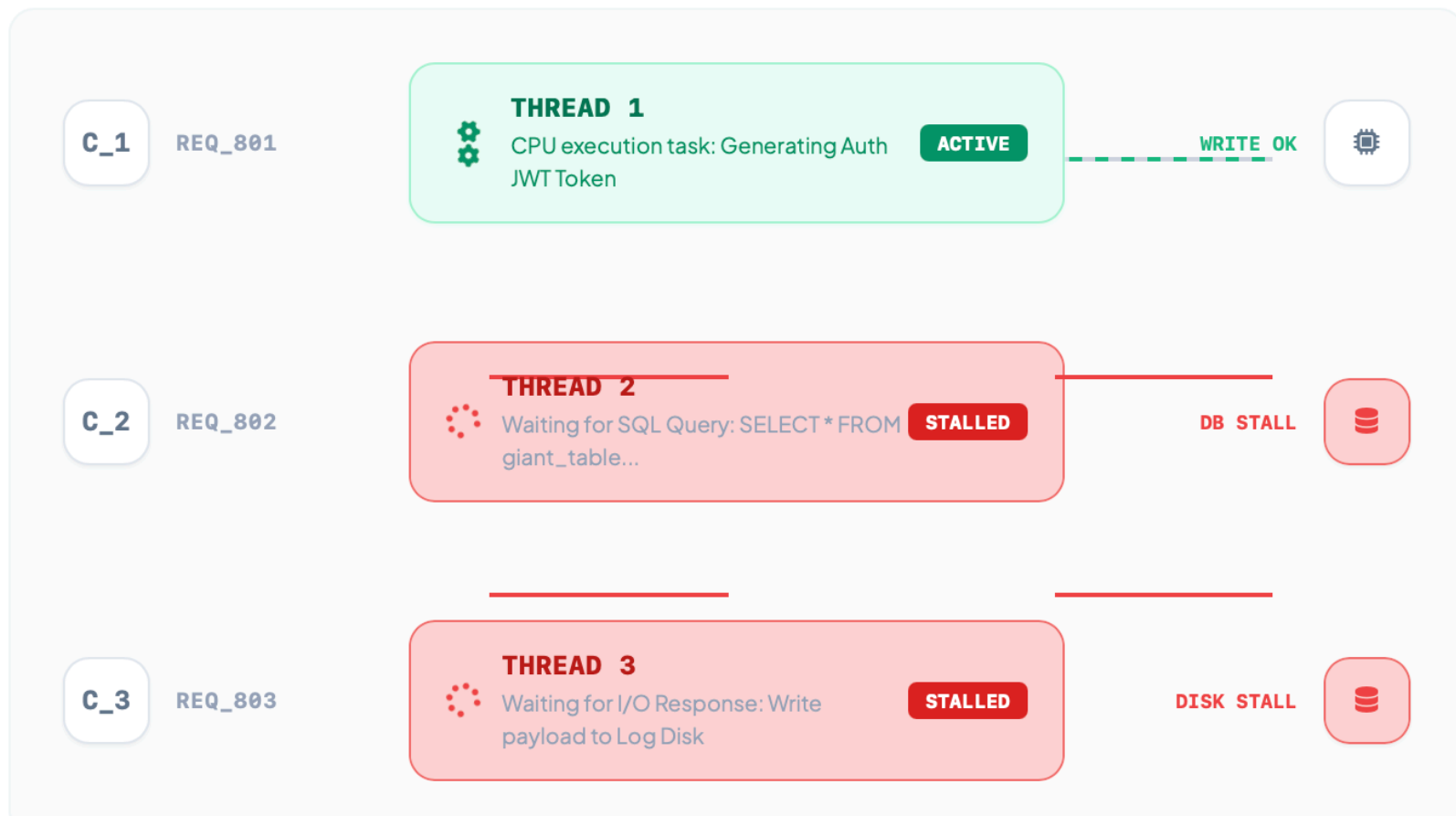
[SYSTEM] Simulation returned to nominal operations.



[SYSTEM] Simulation returned to nominal operations.

[OS_KERNEL] ALERT: Client Thread_2 holding socket waiting for long-running DB transaction.

[OS_KERNEL] ALERT: Client Thread_3 locked on file write system call.

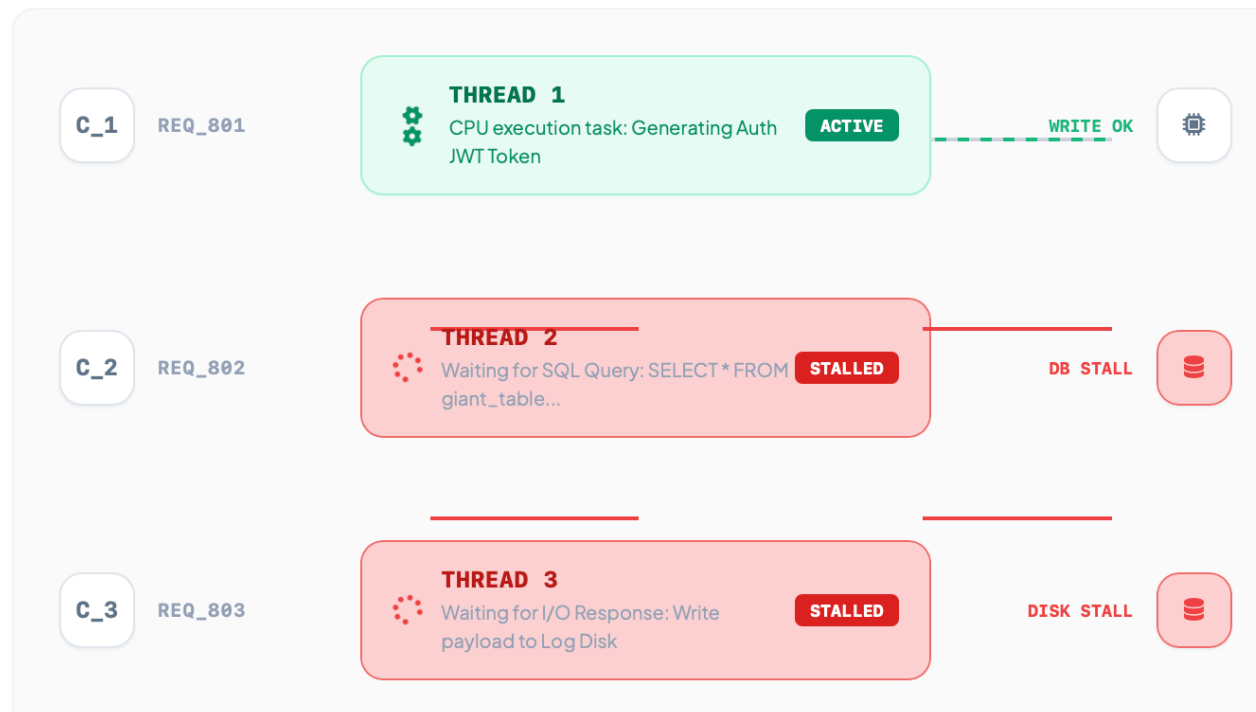


[SYSTEM] Simulation returned to nominal operations.

[OS_KERNEL] ALERT: Client Thread_2 holding socket waiting for long-running DB transaction.

[OS_KERNEL] ALERT: Client Thread_3 locked on file write system call.

[TCP_STACK] INCOMING: Connection from 127.0.0.1:49312. Thread pool exhausted. Appending to backlog queue.



[SYSTEM] Simulation returned to nominal operations.

[OS_KERNEL] ALERT: Client Thread_2 holding socket waiting for long-running DB transaction.

[OS_KERNEL] ALERT: Client Thread_3 locked on file write system call.

[TCP_STACK] INCOMING: Connection from 127.0.0.1:49312. Thread pool exhausted. Appending to backlog queue.

[TCP_STACK] INCOMING: Connection from 127.0.0.1:49314. Queue limits exceeded (Limit: 1). Socket closed.

[OS_KERNEL] Socket connection dropped: ECONNREFUSED

Introduction to FastAPI Architecture

- **What makes FastAPI different?**

- A modern, high-performance web framework for building APIs with **Python** based on standard Python type hints.
- **ASGI Native (Asynchronous Server Gateway Interface)** Unlike older frameworks (Flask/Django) built on WSGI (Web Server Gateway Interface) which are strictly synchronous, FastAPI natively understands both synchronous and asynchronous protocols.

Introduction to FastAPI Architecture

- **The Uvicorn Ecosystem**

- FastAPI is the framework/application layer, but it requires an **ASGI server like Uvicorn** to listen on network ports, handle raw TCP/IP sockets, and translate incoming bytes into Python dictionaries (Scope, Receive, Send).

Under the Hood: Data Validation via Pydantic

- **Parsing and Serialization**

- Traditional frameworks force developers to manually parse JSON strings and validate data types using custom **if/else** statements.
- FastAPI delegates this entire process to **Pydantic**

Under the Hood: Data Validation via Pydantic

- **Strict Type Enforcement**

- When you define a route model using Pydantic, data **validation** happens **before** your endpoint function **executes**.
- If data fails the schema validation, FastAPI automatically generates an **HTTP 422 Unprocessable** Entity response with precise error coordinates, protecting the internal logic from corrupted state inputs.

INCOMING JSON BODY

```
{  
  "robot_id": "robot_1",  
  "power": 95,  
  "active": true  
}
```



CLIENT
(JSON
Payload)



FASTAPI
ROUTER



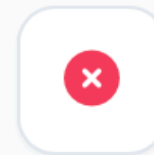
PYDANTIC
VALIDATOR



PYTHON OBJECT
(Type Casted)



ENDPOINT
FUNCTION



422 ERROR
(Bounce Response)

TARGET PYDANTIC SCHEMA

```
class RobotCommand(BaseModel):  
    robot_id: str  
    power: int  
    active: bool
```

INCOMING JSON BODY

```
{
  "robot_id": "robot_1",
  "power": "OVERCHARGE",
  "active": "unknown"
}
```



CLIENT
(JSON
Payload)



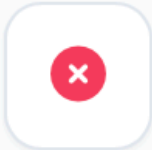
FASTAPI
ROUTER



PYDANTIC
VALIDATOR



PYTHON OBJECT
ENDPOINT
(Type Casted)
FUNCTION



422 ERROR
(Bounce Response)

TARGET PYDANTIC SCHEMA

```
class RobotCommand(BaseModel):
    robot_id: str
    power: int
    active: bool
```

The Transition Point

- **HTTP** is excellent for stateless, intermittent communication but suffers from heavy overhead and connection re-establishment costs for continuous data streams.
- **Traditional Synchronous Architectures** degrade rapidly under massive concurrent request volume due to thread-blocking mechanics.
- **FastAPI** solves validation and parsing problems efficiently, but to truly break the scalability wall of traditional I/O, we must step outside the standard Request-Response paradigm.
- Next Objective Understanding how to move from transient HTTP connections to persistent, bi-directional channels.

fastapi_basic_lab.py

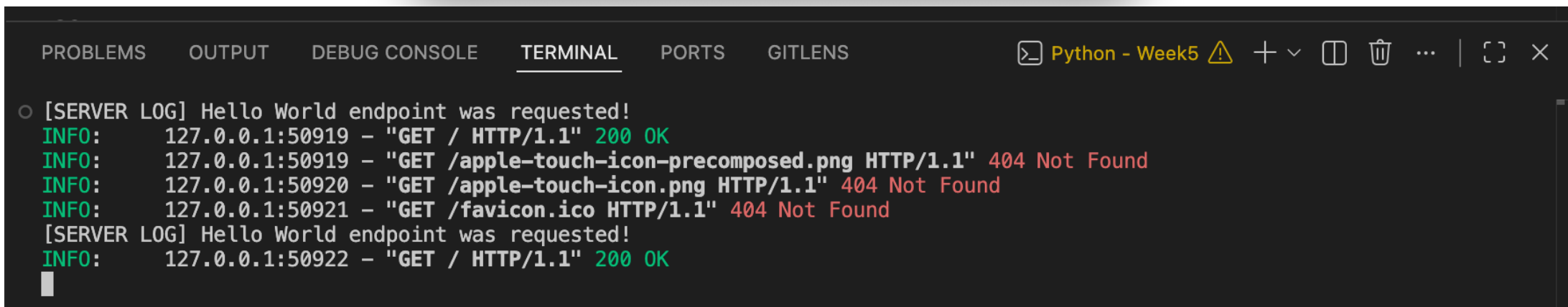
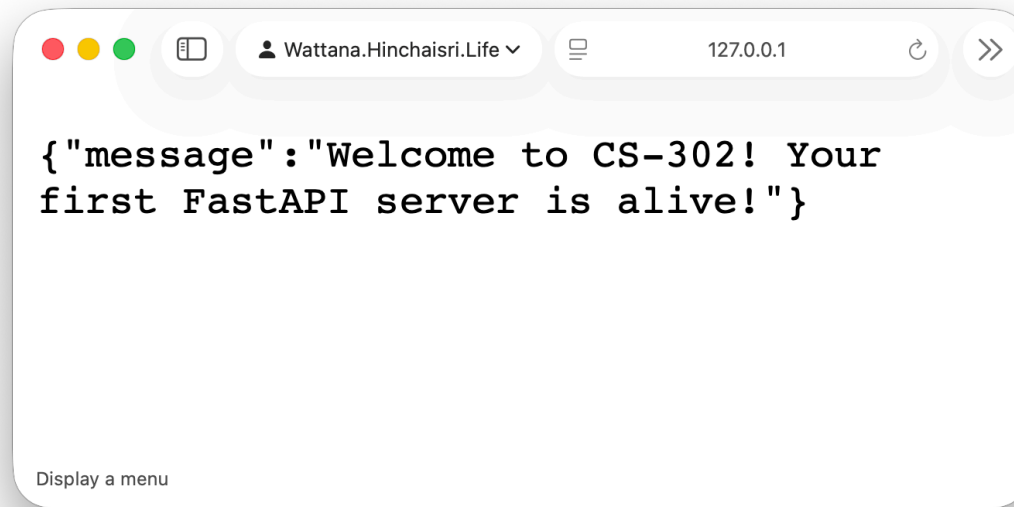
FastAPI lab (Synchronous Thread pool)

```
1  """
2  =====
3  CS-302: Introduction to FastAPI – Basic Lab 01
4  Topic: First Step with FastAPI, Routing, Parameters, & Basic Validation
5  =====
6
7  How to Run This Lab:
8  -----
9  1. Install dependencies (if you haven't yet):
10     $ pip install fastapi uvicorn
11
12  2. Run the development server:
13     $ uvicorn fastapi_basic_lab:app --reload
14
15  3. Open your browser:
16     - Interactive API Docs (Test Area): http://127.0.0.1:8000/docs
17     - Hello World endpoint: http://127.0.0.1:8000/
18  """
```

http://localhost:8000/

```
19
20 from fastapi import FastAPI
21 from pydantic import BaseModel
22
23 app = FastAPI(
24     title="CS-302: Basic FastAPI Lab",
25     description="This is the first step lab for students to understand FastAPI routing and data binding.",
26     version="1.0.0"
27 )
28
29 @app.get("/")
30 def read_root():
31     """
32     Step 1: The simplest GET endpoint.
33     Returns a basic JSON response to confirm the server is running.
34     """
35     print("[SERVER LOG] Hello World endpoint was requested!")
36     return {"message": "Welcome to CS-302! Your first FastAPI server is alive!"}
37
```

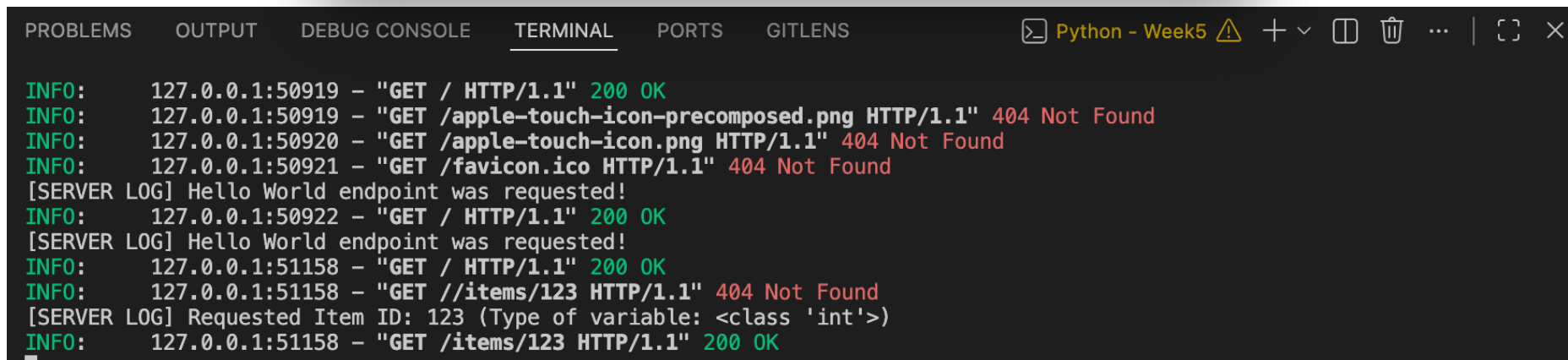
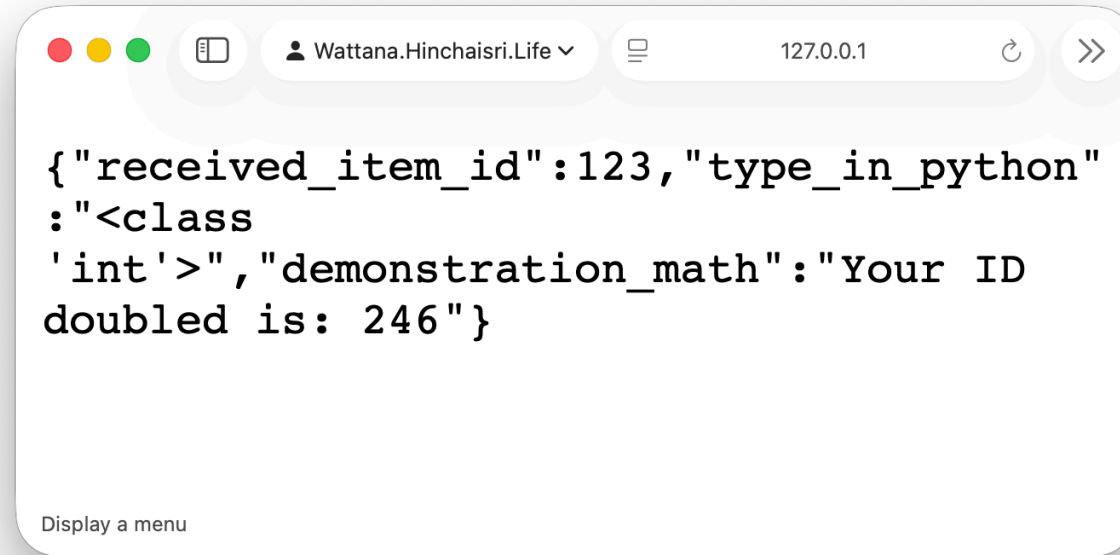
http://localhost:8000/



<http://localhost:8000/items/123>

```
38
39 @app.get("/items/{item_id}")
40 def read_item_by_id(item_id: int):
41     """
42     Step 2: Path Parameters and Type Coercion.
43     FastAPI reads the URL path. Even though 'item_id' arrives as a string (e.g., "/items/123"),
44     FastAPI's type hint 'item_id: int' forces the engine to auto-cast it into a Python Integer.
45     """
46     print(f"[SERVER LOG] Requested Item ID: {item_id} (Type of variable: {type(item_id)})")
47
48     # Notice that we can directly do math operations on item_id because it is already an integer
49     doubled_value = item_id * 2
50
51     return {
52         "received_item_id": item_id,
53         "type_in_python": str(type(item_id)),
54         "demonstration_math": f"Your ID doubled is: {doubled_value}"
55     }
```

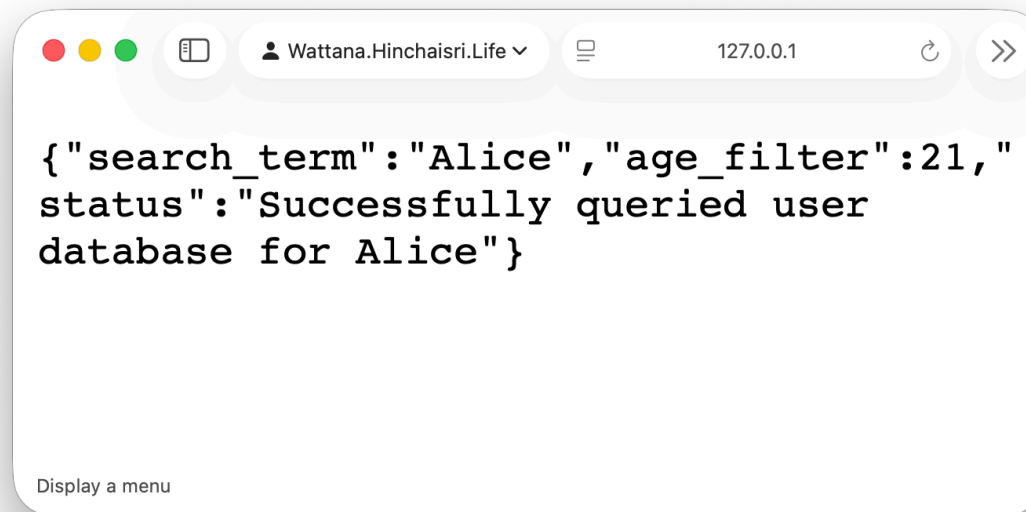
http://localhost:8000/items/123



http://127.0.0.1:8000/users?username=Alice&age=21

```
58 @app.get("/users")
59 def search_users(username: str, age: int = 18):
60     """
61     Step 3: Query Parameters.
62     Access via: http://127.0.0.1:8000/users?username=Alice&age=21
63     'age' has a default value of 18 if the user does not provide it in the URL.
64     """
65     print(f"[SERVER LOG] Searching for user: {username}, Age constraint: {age}")
66     return {
67         "search_term": username,
68         "age_filter": age,
69         "status": f"Successfully queried user database for {username}"
70     }
```

http://127.0.0.1:8000/users?username=Alice&age=21



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS  Python - Week5 ⚠ + ▢ 🗑 ... | [ ] ×

INFO:      127.0.0.1:51158 - "GET //items/123 HTTP/1.1" 404 Not Found
[SERVER LOG] Requested Item ID: 123 (Type of variable: <class 'int'>)
INFO:      127.0.0.1:51158 - "GET /items/123 HTTP/1.1" 200 OK
[SERVER LOG] Hello World endpoint was requested!
INFO:      127.0.0.1:51202 - "GET / HTTP/1.1" 200 OK
[SERVER LOG] Requested Item ID: 123 (Type of variable: <class 'int'>)
INFO:      127.0.0.1:51243 - "GET /items/123 HTTP/1.1" 200 OK
[SERVER LOG] Searching for user: Alice, Age constraint: 21
INFO:      127.0.0.1:51774 - "GET /users?username=Alice&age=21 HTTP/1.1" 200 OK
[SERVER LOG] Searching for user: Alice, Age constraint: 21
INFO:      127.0.0.1:51775 - "GET /users?username=Alice&age=21 HTTP/1.1" 200 OK
```

Pedantic Model

```
73  class SimpleStudent(BaseModel):  
74      """  
75      Step 4: Creating a Pydantic Model.  
76      This model acts as a blueprint/contract for incoming POST data.  
77      """  
78      student_id: str  
79      nickname: str  
80      gpa: float  
81
```

http://localhost:8000/register/student

```
83 @app.post("/register/student")
84 def register_student(student: SimpleStudent):
85     """
86     Step 5: Receiving JSON Data via POST.
87     FastAPI reads the JSON payload from the request body,
88     runs it through the SimpleStudent schema, and casts it into a Python Object.
89     """
90     print(f"[SERVER LOG] New Registration Received!")
91     print(f"[SERVER LOG] ID: {student.student_id}, Name: {student.nickname}, GPA: {student.gpa}")
92
93     # We can access attributes using dot notation directly!
94     is_academic_probation = student.gpa < 2.00
95
96     return {
97         "message": "Student registration data parsed successfully!",
98         "student_object_data": {
99             "id": student.student_id,
100             "name": student.nickname,
101             "current_gpa": student.gpa
102         },
103         "academic_probation_alert": is_academic_probation
104     }
```

http://localhost:8000/register/student

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/register/student' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "student_id": "68123456789",
    "nickname": "Wattana",
    "gpa": 3.0
  }'
```

Request URL

```
http://localhost:8000/register/student
```

Server response

Code	Details
200	Response body <pre>{ "message": "Student registration data parsed successfully!", "student_object_data": { "id": "68123456789", "name": "Wattana", "current_gpa": 3 }, "academic_probation_alert": false }</pre>  Download Response headers <pre>content-length: 171 content-type: application/json date: Tue, 21 Jul 2026 11:35:22 GMT server: uvicorn</pre>

fastapi_async_basic_lab.py

```
1  """
2  =====
3  CS-302: Introduction to FastAPI - Async Basics Lab 02
4  Topic: Understanding async, await, and Non-blocking Cooperative Multitasking
5  =====
6
7  How to Run This Lab:
8  -----
9  1. Run the development server:
10     $ uvicorn fastapi_async_basic_lab:app --reload --port 8000
11
12  2. Open your browser to test endpoints:
13     - Sync Blocking:      http://127.0.0.1:8000/sync-delay
14     - Async Non-Blocking: http://127.0.0.1:8000/async-delay
15     - Concurrent Tasks:  http://127.0.0.1:8000/concurrent-tasks
16  """
```

fastapi_async_basic_lab.py

<http://localhost:8000/sync-delay>

<http://localhost:8000/async-delay>

<http://localhost:8000/concurrent-tasks>

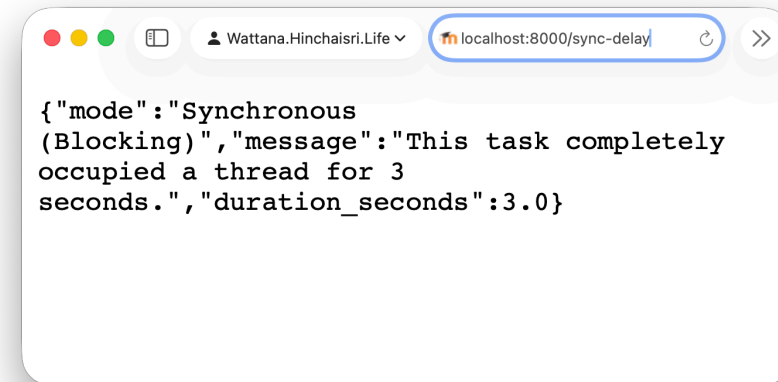
http://localhost:8000/sync-delay

```
28 @app.get("/sync-delay")
29 def sync_delay():
30     """
31     Step 1: Traditional Synchronous Blocking (def)
32     -----
33     - We use 'time.sleep(3)' to simulate a heavy operation (like a slow database query).
34     - Even though FastAPI runs standard 'def' in a thread pool to avoid freezing the main thread,
35       each request still occupies and completely blocks an entire OS thread for 3 full seconds.
36     """
37     start_time = time.time()
38     print("[SERVER LOG] Starting synchronous blocking sleep...")
39
40     # This blocks the thread. No other code can run on this thread during this time.
41     time.sleep(3)
42
43     duration = time.time() - start_time
44     print(f"[SERVER LOG] Finished sync task in {duration:.2f} seconds!")
45
46     return {
47         "mode": "Synchronous (Blocking)",
48         "message": "This task completely occupied a thread for 3 seconds.",
49         "duration_seconds": round(duration, 2)
50     }
51
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS GITLENS

Python - Week5

```
[SERVER LOG] Starting synchronous blocking sleep...
[SERVER LOG] Finished sync task in 3.00 seconds!
INFO: 127.0.0.1:52900 - "GET /sync-delay HTTP/1.1" 200 OK
```

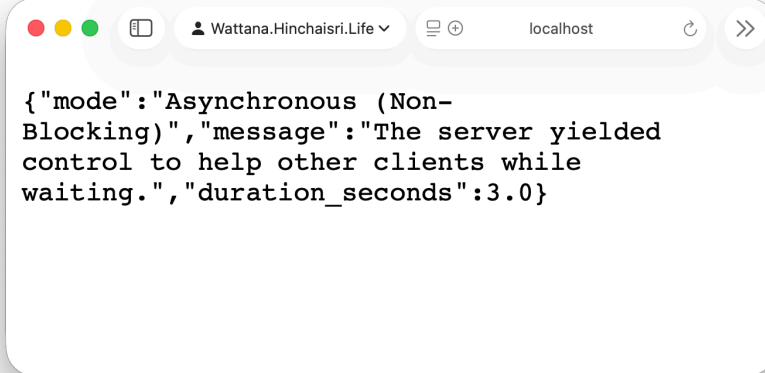


http://localhost:8000/async-delay

```
53 @app.get("/async-delay")
54 async def async_delay():
55     """
56     Step 2: Cooperative Asynchronous (async def)
57     -----
58     - We use 'async def' to run this function directly on the main Event Loop.
59     - We use 'await asyncio.sleep(3)' to simulate waiting for an external response.
60     - Crucial difference: The word 'await' tells the Event Loop, "I am going to wait for 3 seconds.
61       Please feel free to pause me and go handle other incoming user requests in the meantime!"
62     """
63     start_time = time.time()
64     print("[SERVER LOG] Starting cooperative asynchronous sleep...")
65
66     # This does NOT block. It yields control back to the Event Loop.
67     await asyncio.sleep(3)
68
69     duration = time.time() - start_time
70     print(f"[SERVER LOG] Finished async task in {duration:.2f} seconds!")
71
72     return {
73         "mode": "Asynchronous (Non-Blocking)",
74         "message": "The server yielded control to help other clients while waiting.",
75         "duration_seconds": round(duration, 2)
76     }
77
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
[SERVER LOG] Starting cooperative asynchronous sleep...
[SERVER LOG] Finished async task in 3.00 seconds!
INFO: 127.0.0.1:53091 - "GET /async-delay HTTP/1.1" 200 OK
```



```
{
  "mode": "Asynchronous (Non-Blocking)",
  "message": "The server yielded control to help other clients while waiting.",
  "duration_seconds": 3.0
}
```

http://localhost:8000/concurrent-tasks

```
79 @app.get("/concurrent-tasks")
80 async def run_concurrent_tasks():
81     """
82     Step 3: Power of Concurrency (asyncio.gather)
83     -----
84     - What if we have to fetch data from 3 different external APIs, and each takes 2 seconds?
85     - Synchronous way: 2 + 2 + 2 = 6 seconds of total waiting.
86     - Asynchronous way: We can fire all 3 requests at the same time and 'await' them concurrently.
87     - Total waiting time drops to just ~2 seconds (the speed of the slowest task)!
88     """
89     start_time = time.time()
90     print("[SERVER LOG] Starting 3 concurrent tasks...")
91
92     # Define a simple helper async function inside
93     async def fetch_data_from_api(api_name: str, wait_time: int):
94         print(f"🚀 [Task] Starting fetch from {api_name} (takes {wait_time}s)...")
95         await asyncio.sleep(wait_time)
96         print(f"✅ [Task] Finished fetch from {api_name}!")
97         return f"Data from {api_name}"
98
99     # We pack all tasks together and run them in parallel
100     results = await asyncio.gather(
101         fetch_data_from_api("API_Alpha", 2),
102         fetch_data_from_api("API_Beta", 3),
103         fetch_data_from_api("API_Gamma", 1)
104     )
105
106     duration = time.time() - start_time
107     print(f"[SERVER LOG] All concurrent tasks completed in {duration:.2f} seconds!")
108
109     return {
110         "mode": "Concurrent Async Execution",
111         "results_received": results,
112         "efficiency_note": "If executed sequentially, it would have taken 6s (2+3+1). Concurrently, it took only ~3s!",
113         "duration_seconds": round(duration, 2)
114     }
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS GITLENS

Python - Week5

```
🚀 [Task] Starting fetch from API_Alpha (takes 2s)...
🚀 [Task] Starting fetch from API_Beta (takes 3s)...
🚀 [Task] Starting fetch from API_Gamma (takes 1s)...
✅ [Task] Finished fetch from API_Gamma!
✅ [Task] Finished fetch from API_Alpha!
✅ [Task] Finished fetch from API_Beta!
[SERVER LOG] All concurrent tasks completed in 3.00 seconds!
INFO: 127.0.0.1:53450 - "GET /concurrent-tasks HTTP/1.1" 200 OK
```

```
{
  "mode": "Concurrent Async Execution",
  "results_received": [
    "Data from API_Alpha",
    "Data from API_Beta",
    "Data from API_Gamma"
  ],
  "efficiency_note": "If executed sequentially, it would have taken 6s (2+3+1). Concurrently, it took only ~3s!",
  "duration_seconds": 3.0
}
```

Display a menu

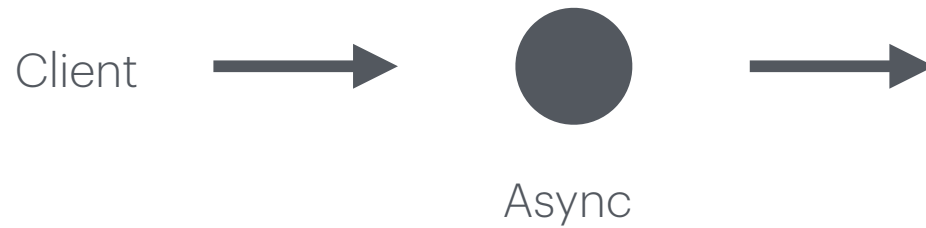
fastapi_async_external_api_lab.py

- Async single-fetch
- Async sequential-fetch
- Async concurrent-fetch

```
# We define highly stable and free public endpoints for our experiments
CAT_FACT_API = "https://catfact.ninja/fact"
BITCOIN_PRICE_API = "https://api.coindesk.com/v1/bpi/currentprice.json"
JOKE_API = "https://official-joke-api.appspot.com/random\_joke"
```

fastapi_async_external_api_lab.py

- Async single-fetch
-



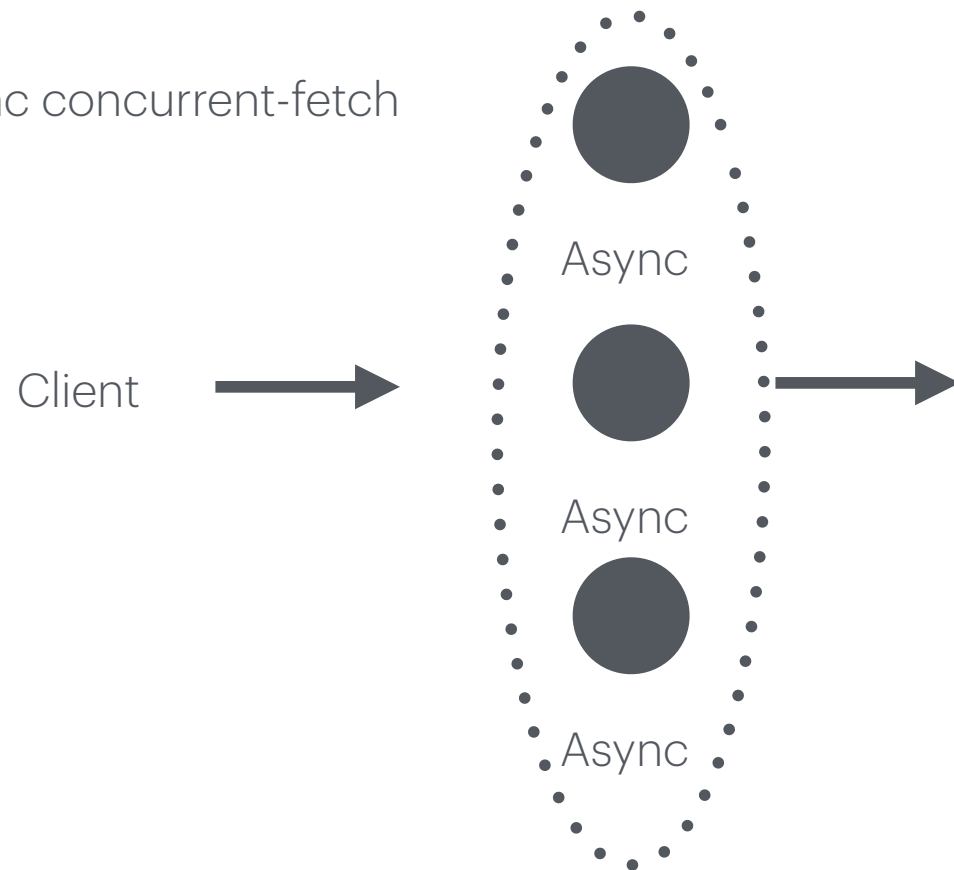
fastapi_async_external_api_lab.py

- Async sequential-fetch
-



fastapi_async_external_api_lab.py

- Async concurrent-fetch



Break

WebSocket

HTTP Request-Response Limitations

- Short Polling Overhead
 - The client **repeatedly shoots** HTTP GET requests at predetermined intervals to fetch updates.
- Long Polling Blockages
 - The server suspends HTTP response delivery, keeping the TCP connection open until fresh data is available.

What is WebSocket?

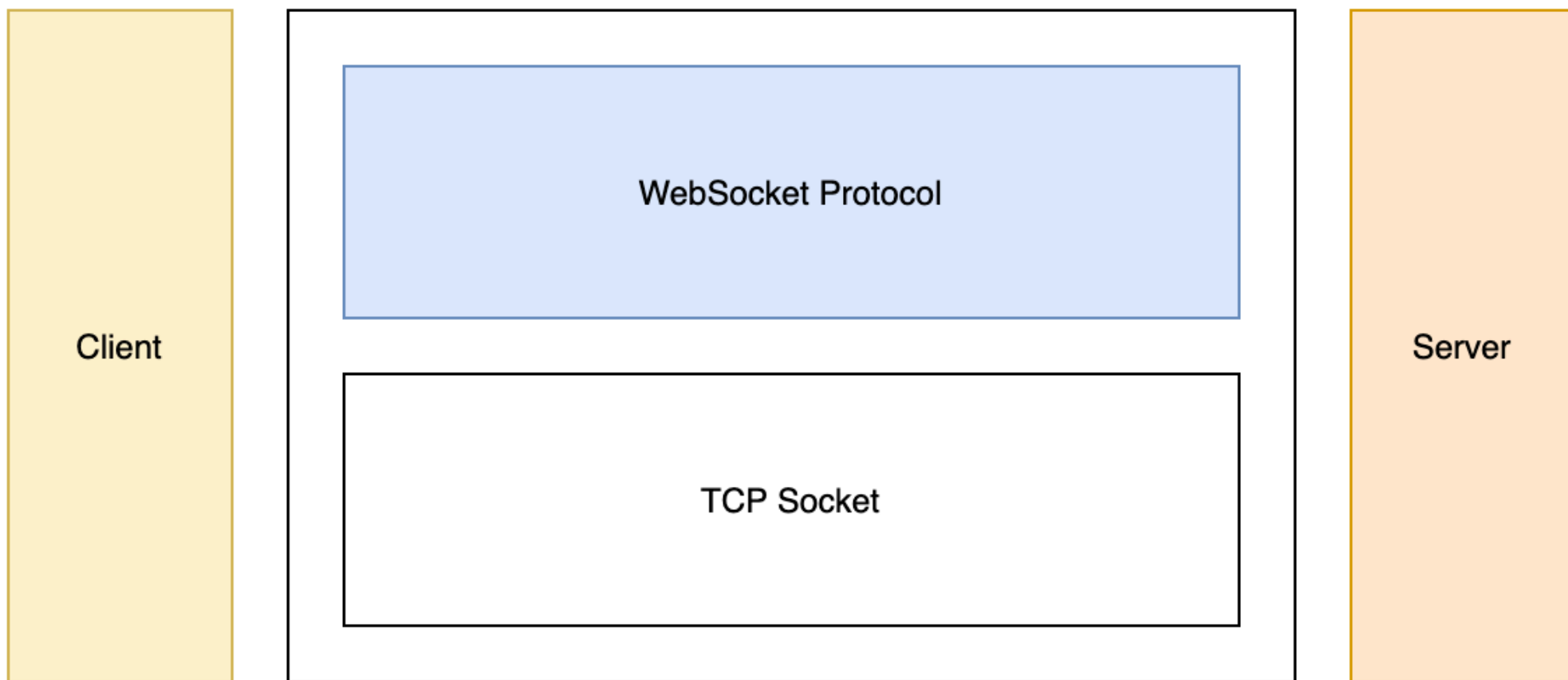
- Full-Duplex TCP Channels
 - Standardized as **RFC 6455 in 2011**, WebSocket is an entirely separate network protocol designed to **establish persistent, bi-directional** communication channels.
 - Both **client** and **server** send payload frames independently at any moment over a single, **active TCP pipe**. It dramatically decreases overhead by avoiding repeated HTTP handshakes.

HTTP vs. WebSockets

- Standard HTTP (1.1 / 2)
 - Ephemeral (Opens & closes per request)
 - Unidirectional (Request -> Response)
 - High (800 bytes - 2KB per transaction)
 - Unsupported (Requires long-poll tricks)
- WebSocket Protocol
 - Persistent (Stays open continuously)
 - Full-Duplex (Simultaneous communication)
 - Extremely Low (2 - 10 bytes after handshake)
 - Native (Server pushes data instantly)

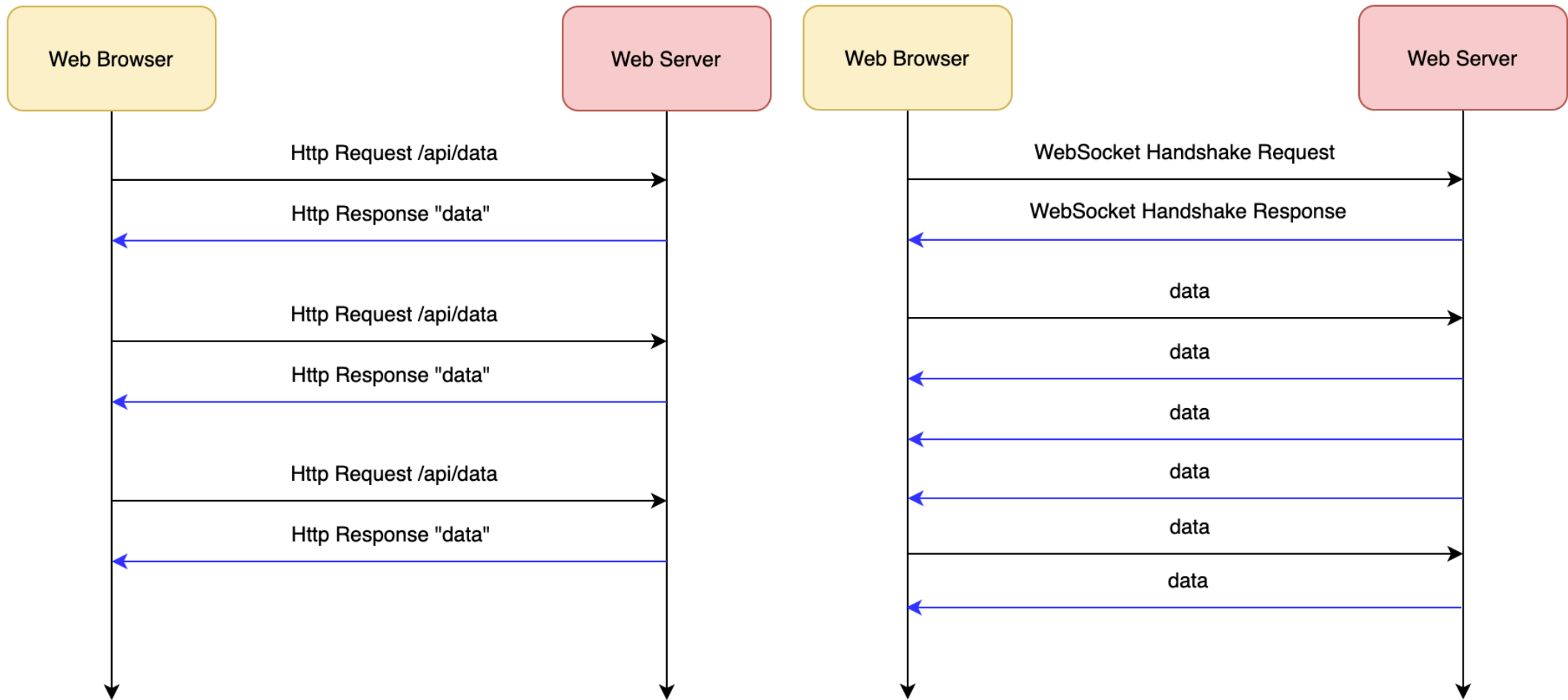
The WebSocket Handshake

- Understanding how standard web infrastructure initiates, upgrades, and secures the transition to WebSockets.



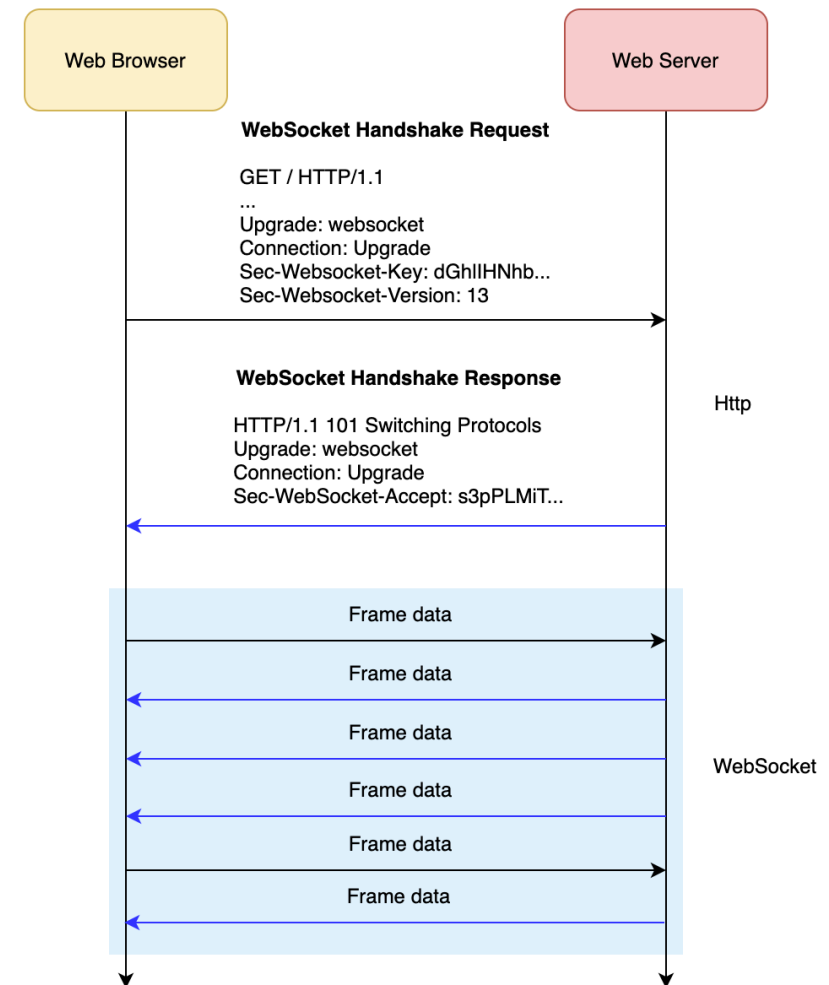
Upgrading the Connection

- The Handshake Flow
 - The communication begins inside standard Web architecture by using established HTTP routes (ports 80 / 443).
 - The client initiates an ordinary HTTP GET request containing explicit Upgrade declarations. The server responds with 101 status code, seamlessly transitioning the underlying raw socket to WebSocket.



Core Handshake Headers

- **Upgrade: websocket** — Expressly instructs the server to transition the communication protocol layer from HTTP.
- **Connection: Upgrade** — Indicates that the client intends to reuse the active connection for the upgraded format.
- **Sec-WebSocket-Key** — A random Base64 security string generated to prevent proxy caches from serving old connections.
- **Sec-WebSocket-Version** 13 — Identifies the protocol standards version expected by client runtime.



The Browser Client API

- 1. Connect
Open connections easily using native browser constructors
- 2. Communicate
Listen and push payload events instantly without loops
- 3. Lifecycle
Register connection closing and error events

```
const ws = new  
WebSocket('ws://host');
```

```
ws.onmessage = (e) =>  
  console.log(e.data);  
ws.send('Data');
```

```
ws.onclose = () =>  
  console.log('Closed');
```

Scaling with Async Servers

- Transitioning from blocking synchronous networks to highly concurrent event-driven WebSocket architectures.

Sync vs. Async Scalability

- Thread-per-Connection (Blocking)
Max ~2,000 active connections
2,000 Connections (OS Thread Limit)

2,000

- Asynchronous Event-Loop (Non-Blocking)

100,000

Implementing a Connection Manager in FastAPI

```
1 import asyncio
2 from typing import List
3 from fastapi import FastAPI, WebSocket, WebSocketDisconnect
4 from fastapi.responses import HTMLResponse
5
6 app = FastAPI(title="FastAPI Async WebSocket Lab")
7
```

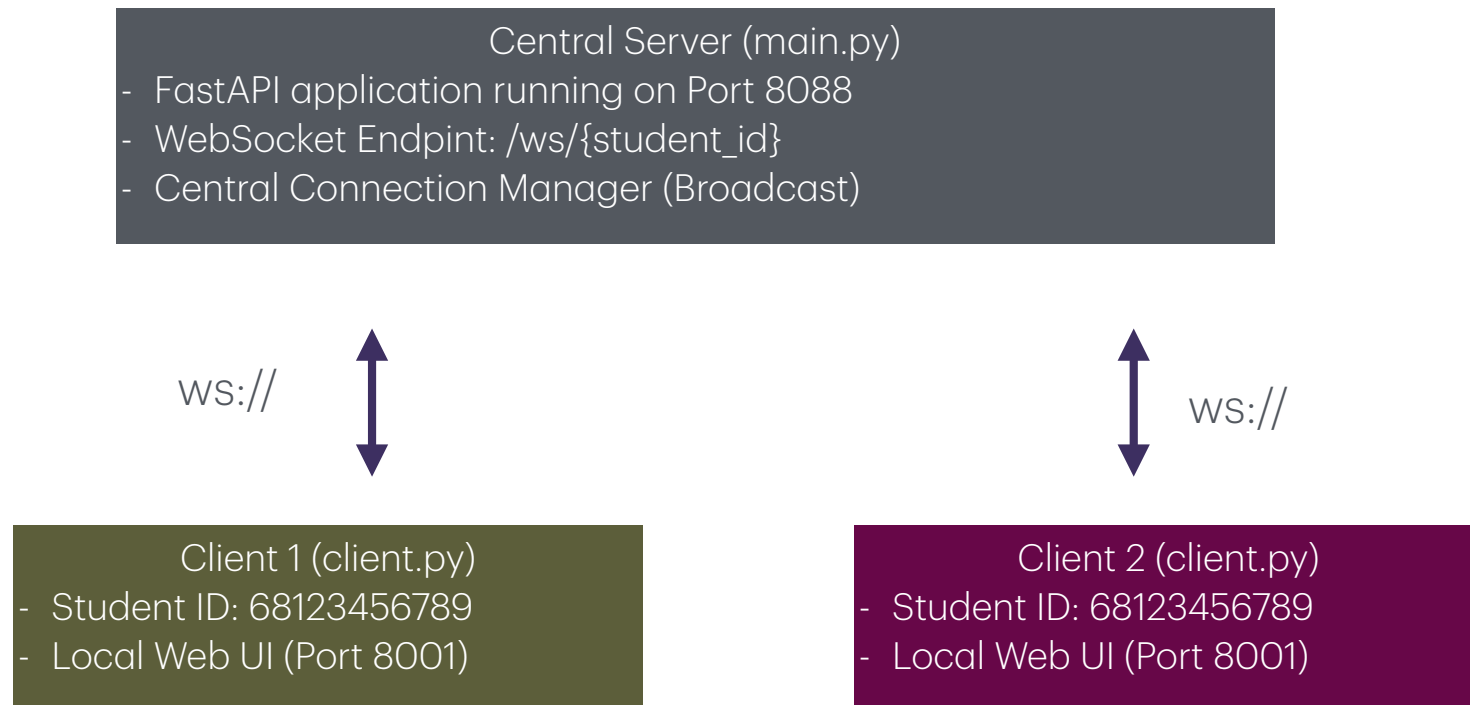
```
65 # -----
66 # WebSocket Connection Manager (ตัวจัดการกระจายข้อความ)
67 # -----
68 class ConnectionManager:
69     def __init__(self):
70         # เก็บรายการ WebSocket Connection ที่ยังคงต่ออยู่
71         self.active_connections: List[WebSocket] = []
72
73     async def connect(self, websocket: WebSocket):
74         await websocket.accept()
75         self.active_connections.append(websocket)
76
77     def disconnect(self, websocket: WebSocket):
78         self.active_connections.remove(websocket)
79
80     async def broadcast(self, message: str):
81         # Async Loop ส่งข้อความหาทุกคนในวงพร้อมกัน
82         for connection in self.active_connections:
83             await connection.send_text(message)
84
85 manager = ConnectionManager()
```

```
88 # -----
89 # WebSocket Endpoint
90 # -----
91 @app.websocket("/ws/{client_id}")
92 async def websocket_endpoint(websocket: WebSocket, client_id: int):
93     # 1. ยอมรับการเชื่อมต่อ (Handshake)
94     await manager.connect(websocket)
95     await manager.broadcast(f"[System]: Client #{client_id} เข้าร่วมห้องแชท")
96
97     try:
98         while True:
99             # 2. รอรับข้อมูลจาก Client แบบ Non-blocking (Await)
100             data = await websocket.receive_text()
101
102             # 3. ส่งข้อมูลกระจายให้ทุกคน (Broadcast)
103             await manager.broadcast(f"Client #{client_id}: {data}")
104
105     except WebSocketDisconnect:
106         # 4. เมื่อ Client ปิดหน้าจอ หรือเน็ตหลุด
107         manager.disconnect(websocket)
108         await manager.broadcast(f"[System]: Client #{client_id} ออกจากห้องแชท")
```

Break

Multi-Client Chat Application

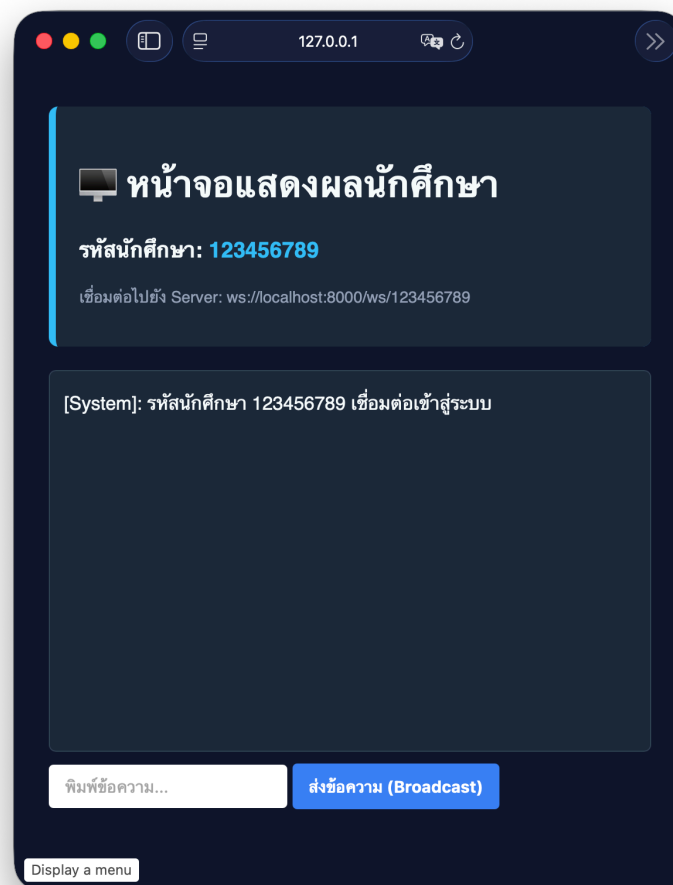
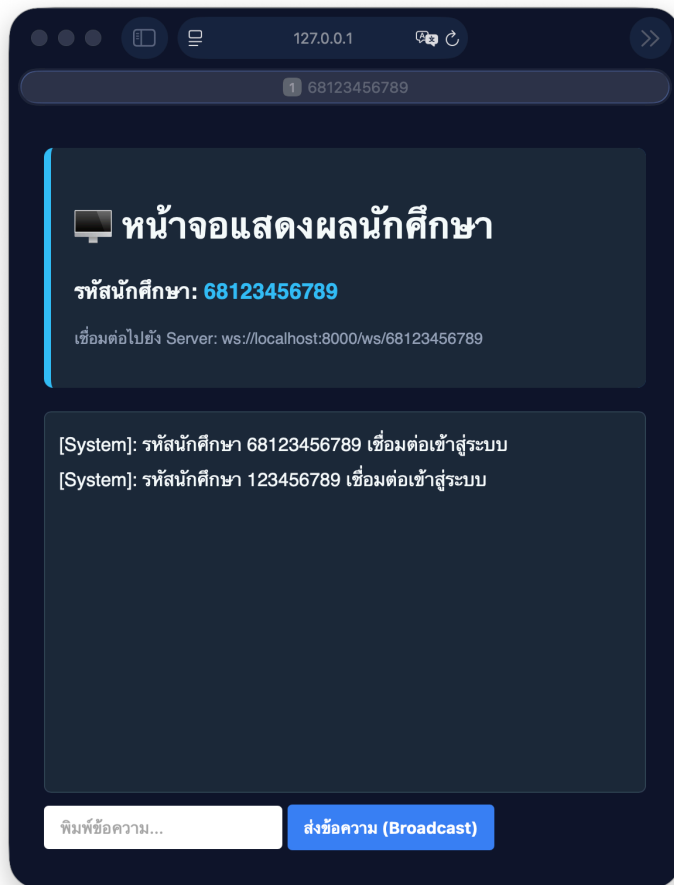
Real-Time Chat System Architecture



Server Side (main.py)

- ConnectionManager Class:
 - Stores connected WebSocket instances in a dictionary using student_id as the key (Dict[str, WebSocket]).
 - Manages client lifecycle: connect(), disconnect(), and broadcast().
- broadcast() Function
 - Asynchronously loops through all active client connections
 - Pushes messages instantly to all connected screens without requiring HTTP polling.

Client Side (client.py)



Why WebSockets & FastAPI?

- **Full-Duplex Communication** Simultaneous, two-way data flow over a single TCP connection.
- **Low Latency** Eliminates the heavy overhead of repeated HTTP request/response headers.
- **Asynchronous Efficiency** Built on Python's asyncio via FastAPI, handling multiple connected screens effortlessly.
- **Scalable Identification:** URL path parameters (`/ws/{student_id}`) allow seamless client identification and tracking.

🚀 Mission Control Dashboard

Active Rockets: 2 | Arena: 800 x 600

123456



987



🚀 Rocket Controller

ID: 123456

เชื่อมต่อไปยัง: ws://localhost:8088/ws/123456

ใช้ปุ่มด้านล่าง หรือกดลูกศร (← → ↑) บนคีย์บอร์ดเพื่อขับเคลื่อน

← Left

Right →

🔥 THRUST 🔥